

INDEPENDENT AI-ASSISTED CODE SECURITY REVIEW

USD8 Core Security Audit Report

Smart contracts and offchain insurance settlement

AUDIT OPINION

**PRODUCTION DEPLOYMENT BLOCKED PENDING
REMEDiation**

Review date

13 July 2026

Review target

Git commit eda7a22 plus the current uncommitted worktree

Prepared by

Codex - adversarial smart-contract and offchain review



Contents

The report is organized by exploit impact and remediation priority.

Executive summary	3
Risk interpretation	3
Priority actions	3
Scope and methodology	4
Review target	4
Method	4
Limitations	4
Detailed findings	5
C-01 - Legacy proxies are storage-incompatible with current implementations	5
H-01 - The final claimant unfreezes pools before payout	6
M-01 - Silent RPC log truncation corrupts eligibility and score	7
M-02 - Capped no-show claims suppress honest payouts without consuming score	7
M-03 - Score spent earlier in the opening block can be reused	8
M-04 - Dust registrations can force settlement past its deadline	8
M-05 - ERC-4626 donation inflation defeats the strategy loss tolerance	9
M-06 - Treasury rotation fragments backing from global liabilities	9
L-01 - Savings safety depends on a dead-share deployment convention	10
L-02 - Revenue hooks are credited when a receiver pulls nothing	10
Trust model and assurance observations	11
Verification results	12
PoC inventory	12
Repository state	12
Remediation roadmap	13
Recommended retest package	13

Executive summary

The USD8 Core working tree is **not ready for production deployment or upgrades**. The review confirmed one Conditional Critical storage-migration defect, one High-severity cross-contract callback vulnerability, six Medium-severity settlement, strategy, and migration issues, and two Low-severity lifecycle or accounting defects.

AUDIT OPINION

Block production until C-01 and H-01 are closed. Treat M-01 through M-04 as one settlement-integrity workstream before insurance claims are enabled.

Conditional Critical		1
High		1
Medium		6
Low		2

ID	Severity	Finding	Activation condition
C-01	Conditional Critical	Legacy proxy storage incompatibility	Direct legacy-layout implementation upgrade
H-01	High	Last claimant unfreezes pool before payout	Callback-capable external interaction
M-01	Medium	RPC log truncation corrupts eligibility or score	Provider cap below assumed 10,000
M-02	Medium	Capped no-show claim suppresses honest payouts	Permissionless claim plus reusable score
M-03	Medium	Same-block prior score spend is ignored	Prior finalization and next open in one block
M-04	Medium	Dust registrations can defeat settlement deadline	Sufficient sybils and history depth
M-05	Medium	ERC-4626 donation inflates allowed loss	Low-supply inflation-vulnerable vault
M-06	Medium	Treasury switch fragments backing	Authority changed without reserve migration
L-01	Low	Savings lifecycle depends on dead-share seed	Zero live supply or pre-TVL revenue
L-02	Low	Non-pulling receiver is credited	Misconfigured future receiver

Risk interpretation

The exact fresh deployment configuration contains an empty insured-token registry and does not immediately expose a privilege-free drain. That is not a safe operating state: the High and several Medium findings become reachable when the system performs its intended functions, including listing assets, accepting claims, settling incidents, allocating strategy capital, or rotating the Treasury.

C-01 is irrelevant to fresh current-layout deployments. It is a deployment blocker for any USD8, SavingsUSD8, or SingleAssetCoverPool proxy that retains the pre-6d44001 legacy layout. Deployment date alone is not dispositive. First inventory onchain implementation history and storage layout.

Priority actions

- 1 Identify every deployed proxy and beacon implementation before authorizing any upgrade.
- 2 Keep incidents active throughout all finalization callbacks and external interactions.
- 3 Close settlement-integrity, strategy, Treasury-migration, and release-assurance gaps before enabling production operation.

Scope and methodology

Review target

The review targeted Git commit **eda7a22** and the current worktree at `/Users/rickchen/codes/usd8Codes/usd8core`. The uncommitted deposit-cap changes in `SingleAssetCoverPool` and its tests were included. Existing README changes were preserved.

Area	Reviewed components
Core token and governance	RegistryManaged, Registry, USD8, Treasury
Savings and coverage	SavingsUSD8, SingleAssetCoverPool, DefiInsurance
Strategies	ERC4626Strategy and Treasury strategy integration
Offchain settlement	Event acquisition, history reconstruction, scoring, allocation, Merkle construction, CLI
Deployment and testing	Deployment scripts, upgrade tests, unit tests, integration tests, fuzzing, invariants, CI

Method

- Manual review of authorization, accounting, storage layout, upgradeability, incident state transitions, and trust boundaries.
- Adversarial tracing of cross-contract callbacks, token hooks, delayed exits, settlement timing, and governance configuration changes.
- Independent review of offchain history completeness, score snapshots, allocation mathematics, Merkle inputs, and deadline behavior.
- Executable reproductions or quantified adversarial analysis cover every ranked finding, including both Low findings.
- Regression, integration, build, coverage, stateless fuzz, stateful invariant, dependency, size, lint, and formatting checks.
- Historical source reconstruction around commit 6d44001 and live-state sanity checks for configured ERC-4626 vault assumptions.

Limitations

The production TEE signer, depeg-detection service, signing-key custody, enclave build, oracle adapters, booster source, operational monitoring, and final deployment records are not present in this repository. Those components form a material portion of the settlement root of trust and were not auditable. External tokens and vaults were reviewed only at their integration boundary, not as standalone protocols.

POINT-IN-TIME ASSURANCE

This report is a point-in-time code review, not a guarantee that all defects are known. Any material code, configuration, dependency, deployment, signer, or governance change requires focused re-review.

Detailed findings

CONDITIONAL CRITICAL

STATUS: OPEN

C-01 - Legacy proxies are storage-incompatible with current implementations

Affected. [src/RegistryManaged.sol:41](#), [src/USD8.sol:48](#), [src/SavingsUSD8.sol:54](#), and [src/SingleAssetCoverPool.sol:86](#).

Before commit **6d44001**, the Managed base stored authority in storage slot 0 followed by a 50-slot gap. Child state therefore began at slot 51. The current RegistryManaged base moved the Registry reference into ERC-7201 namespaced storage, allowing each child's ordinary state to begin at slot 0. A direct upgrade does not migrate data; it causes the new implementation to interpret legacy values under incompatible meanings.

Contract	Legacy state	Current interpretation and result
USD8	authority at slot 0; treasury at slot 51	Old authority is read as Treasury; registry is zero; true Treasury is stranded. Balances survive, but mint, burn, authorization, and ordinary UUPS recovery fail.
SavingsUSD8	authority/gap at slots 0-50; profit state at 51+	Profit and accounted-asset fields read authority or gap values; assets, vesting, authorization, and upgradeability are corrupted.
SingleAssetCoverPool	custom asset/share/reward mappings at 51+	Current fields start at slot 0 and OZ token/vault state uses namespaces. Legacy shares and deposited assets are no longer represented by the current accounting model.

Impact. A direct legacy-to-current upgrade can cause protocol-wide loss of control, unusable minting or redemption, corrupted savings accounting, or inaccessible LP positions. The ordinary UUPS path can become unusable after the bad upgrade.

Precondition. The affected proxy or beacon must retain the pre-6d44001 legacy layout and be upgraded directly to a current implementation. Deployment date alone is not dispositive; current-layout deployments are not affected.

Evidence. A passing legacy-proxy PoC, `test_LegacyManagedToNamespacedBaseUpgradeBricksProxy`, reproduced the storage reinterpretation and authorization brick. Existing upgrade coverage tests only current-layout-to-current-layout upgrades.

Recommendation

- Inventory the implementation address, code hash, deployment block, initializer version, and storage layout of every live proxy and beacon before any upgrade.
- Do not directly upgrade a legacy-layout proxy. For USD8 and Savings, use a bridge that preserves the legacy layout, copies Registry into ERC-7201 storage, and migrates child fields into their current ordinary layout.
- Set Savings accounted assets from the verified live asset balance during migration rather than copying an assumed value.
- Treat each legacy-layout pool proxy as a state migration: deploy a fresh pool and reconstruct positions from a reviewed snapshot or Merkle process. The beacon holds no pool accounting state and cannot reconstruct user mappings.
- Archive historical storage layouts and add fork-based legacy-to-bridge-to-current upgrade tests to CI.

SEVERITY NOTE

Rated Conditional Critical because the failure is protocol-wide and potentially irreversible. A methodology that always discounts authorized upgrade triggers may label it High; the required operational response is the same: block the upgrade until storage provenance is proven.

HIGH

STATUS: OPEN

H-01 - The final claimant unfreezes pools before payout

Affected. [DefilInsurance.finalizeClaim](#), [refund interaction](#), [pool payout](#), and [SingleAssetCoverPool.completeRedeem](#).

finalizeClaim marks the claim resolved and decrements the incident's unresolved count before refunding excess escrow, burning boosters, paying the pool, and recording score use. When the claim is the final unresolved claim, the incident becomes externally observable as inactive while its payout is still in progress.

Exploit sequence

- 1 The attacker's pool exit has matured and the attacker is the final unresolved claimant.
- 2 finalizeClaim decrements unresolved claims to zero.
- 3 An over-escrow refund through a callback-capable listed token invokes attacker-controlled code.
- 4 The callback enters completeRedeem. Registry payoutIncidentActive now returns false, so the pool allows redemption at the pre-loss exchange rate.
- 5 finalizeClaim resumes and pays the insurance loss into the pool after the attacker has exited. Remaining LPs absorb the loss.

Impact. A claimant can escape the intended payout haircut and transfer the incident loss to remaining LPs. The demonstrated scenario began with two LPs holding 100 units each; the attacker exited with 100 before an 80-unit payout, leaving the victim's shares backed by only 20 units.

Why nonReentrant is insufficient. The guard protects reentry into DefilInsurance. The callback enters a different contract, SingleAssetCoverPool, whose own guard has no knowledge that DefilInsurance is mid-finalization.

Precondition. A final unresolved claimant with a matured LP exit and a callback surface in a listed or upgradeable insured token, booster, refund, or equivalent external interaction. Standard current assets reduce immediate reachability but do not enforce the invariant.

Evidence. The passing test test_LastClaimRefundCallbackEscapesPoolHaircut reproduced the full cross-contract callback and value transfer.

Recommendation

Keep the incident active until all refunds, burns, payouts, and score updates have completed. Move the unresolved decrement to the end of successful finalization, or introduce an explicit payoutInProgress flag that the Registry includes in its pool-freeze predicate. Add regression tests using callback tokens and multiple external-interaction orderings.

MEDIUM

STATUS: OPEN

M-01 - Silent RPC log truncation corrupts eligibility and score

Affected. [getLogsChunked/getLogsBisect](#), [minBalanceOver](#), and [tokenBlockIntegral](#).

The event reader assumes that any RPC result containing fewer than 10,000 logs is complete. A provider that silently caps results below 10,000 returns a truncated response that is accepted without subdivision or an error. Blockscout, for example, documents a 1,000-result `eth_getLogs` limit and pagination: [official Blockscout RPC documentation](#).

Impact. Omitted transfer events can inflate minimum balances or score integrals. The settlement service may sign an incorrect root or direct payout, overpaying an attacker up to the configured claim cap. A dispute watcher is only a backstop and does not make the signed calculation correct.

Evidence. A reproduction generated 1,000 harmless self-transfers followed by the real outflow. The true minimum balance was zero; the production calculator returned 100 because the outflow fell beyond the provider's lower cap.

Recommendation

- Use a controlled archival or indexed source with explicit, tested pagination semantics.
- Fail closed unless range completeness can be proven; a universal guessed result cap cannot prove completeness.
- Cross-check settlement-critical histories against an independent RPC or indexer.
- For dense single-block ranges, enumerate transaction receipts rather than accepting an ambiguous capped log response.
- Continuously test configured production providers with synthetic ranges above and below documented caps.

MEDIUM

STATUS: OPEN

M-02 - Capped no-show claims suppress honest payouts without consuming score

Affected. [weighted allocation](#), [per-claim cap](#), and [withdrawNonFinalizedClaim](#).

All positive-loss claimant scores are included in the allocation denominator. Each resulting payout is then clipped to escrow or eligibility, but clipped excess is not redistributed. Score and boosters are only consumed if the claimant voluntarily finalizes.

Exploit. A high-score attacker submits a dust-sized claim, captures almost all weighted allocation, has it clipped to almost zero, refuses to finalize, and later recovers the full escrow. Honest allocation is suppressed while the attacker's score and boosters remain reusable.

Evidence. In a production-code settlement PoC, a one-wei attacker claim with score 2,000,000 reduced a score-200 honest claimant's payout from about USD 80 to about USD 0.01. The attacker's economically unused allocation was not redistributed.

Recommendation

- Use iterative capped weighted water-filling so cap excess is redistributed to claims that still have capacity.
- Remove nonfinalizer optionality: reserve score at settlement, escrow booster capacity, make proven finalization permissionless to a fixed owner, or impose a meaningful nonfinalization bond or penalty.
- Test dust claims, cap saturation, rounding, multiple capped claimants, and deliberate nonfinalization.

IMPORTANT

Water-filling fixes discarded allocation, but it does not alone prevent a large capped claimant from reserving capacity and refusing to finalize. Allocation and capacity consumption must be repaired together.

MEDIUM

STATUS: OPEN

M-03 - Score spent earlier in the opening block can be reused

Affected. [settlement score snapshot](#) and [score consumption recording](#).

The canonical settlement calculation reads prior score spending at `openBlock - 1`. If incident A is finalized and records score, and incident B opens later in the same transaction or block, B's eventual calculation deliberately queries the state before A's recorded spend.

Impact. The claimant can reuse score already consumed by a prior incident, increasing allocation at the expense of other claimants. A passing test reused 500 score across incidents.

Precondition. The next incident opens later in the same block as the prior incident's final score recording. Separate transactions in one block are sufficient; reentrancy during finalization is not required.

Recommendation

Read score-spent state at the end of `openBlock`. A newly opened incident cannot finalize in its opening block, so the end-of-block snapshot includes all prior same-block consumption without including consumption from the incident being settled.

MEDIUM

STATUS: OPEN

M-04 - Dust registrations can force settlement past its deadline

Affected. [joinClaim](#), [cancellation filtering](#), [score reads](#), and [history scans](#).

`joinClaim` accepts any nonzero escrow and has no global or per-identity registration bound. Sybils can create unlimited live claims, while canceled claims can be refilled. The offchain pipeline performs $O(N^2)$ cancellation filtering, launches unbounded score-read promises for every registration including canceled ones, and performs chunked lifetime transfer-history scans for every live claimant.

Impact. A low-capital attacker can create enough computation and RPC work to prevent a root from being generated before the fixed three-day submission deadline, voiding otherwise valid coverage. Under a one-year, two-score-token scenario, one live claimant can require roughly one thousand RPC calls.

Cost and constraints. The attacker pays claim-registration gas and dust escrow. Cancellation allows escrow recycling; live sybils remain unlimited. Provider throttling and lower log caps amplify the liveness failure.

Recommendation

- Require a meaningful minimum claim value or bond and a nonrefundable anti-spam component.
- Cap or rate-limit incident registrations and define an operational maximum settlement workload.
- Replace $O(N^2)$ cancellation lookup with a set and avoid score/history work for canceled registrations.
- Use bounded concurrency, caching, batching, and a preindexed score service.
- Add a narrowly governed, observable deadline-extension mechanism for objectively overloaded settlement, without giving administrators arbitrary payout control.

MEDIUM

STATUS: OPEN

M-05 - ERC-4626 donation inflation defeats the strategy loss tolerance

Affected. [ERC4626Strategy.deploy](#).

The strategy accepts a shortfall up to `vault.convertToAssets(1) + 1`, where 1 is one atomic share base unit. The allowed loss rises with that base unit's asset value. A low-supply, donation-vulnerable ERC-4626 vault lets an attacker inflate both the base-unit value and the adapter's tolerance.

1. Attacker seeds 1 asset unit and receives 1 atomic share base unit.
2. Attacker donates 50 units directly to the vault.
3. Treasury deploys 100 units; rounding mints only 1 additional share base unit.
4. Received value is about 75; a loss of about 25 passes a tolerance near 76.
5. Attacker redeems the inflated original share base unit and captures the rounding loss.

Evidence. The passing donation-inflation PoC produced a roughly 25 USDC loss on a 100 USDC Treasury deployment while the current tolerance check passed.

Current configuration. At Ethereum block 25,522,393, the hardcoded Aave and Morpho targets had mature supply and negligible atomic-share-base-unit values, so those targets did not satisfy the exploit precondition. The generic adapter nevertheless permits governance to approve other ERC-4626 vaults.

Recommendation

- Enforce configured absolute and basis-point maximum loss in asset units.
- Require a minimum number of shares based on `previewDeposit` plus an explicit slippage bound.
- Reject vaults with tiny supply or an excessive atomic-share-base-unit value.
- Only approve reviewed vaults with virtual-share/decimal-offset defenses and sufficient established liquidity.

MEDIUM

STATUS: OPEN

M-06 - Treasury rotation fragments backing from global liabilities

Affected. [USD8.setTreasury](#), [Treasury.redeemUSD8](#), and [local reserve accounting](#).

`setTreasury` changes the only mint and burn authority without moving reserves or assigning liabilities. Each Treasury accounts only for its own reserves, while redemption uses the global USD8 total supply. After rotation, legacy supply remains globally visible but is backed in a Treasury that is no longer allowed to burn.

Confirmed consequences. The old funded Treasury cannot complete redemption because USD8 rejects its burn. An empty new Treasury can burn a holder's USD8 for zero assets when `minUsdcOut` is zero. New depositors into the new Treasury are diluted by global supply whose backing remains in the old Treasury.

Evidence. Two passing PoCs reproduced zero-asset burning after rotation and dilution of a new depositor by supply backed in the old Treasury. Existing tests assert authority rotation but do not assert preservation of reserve-to-liability invariants.

Recommendation

Prefer a stable Treasury proxy. If rotation is required, expose only a paused atomic migration that withdraws all strategies, transfers every reserve and accounting obligation, verifies global liabilities against destination backing, and switches mint/burn authority as the final step. Disallow ordinary `setTreasury` calls once supply is nonzero.

LOW

STATUS: OPEN

L-01 - Savings safety depends on a dead-share deployment convention

Affected. [receiveProfitDistribution](#) and [deployment seed](#).

Without a permanent seed, all live shares can be redeemed while profit remains pending. With assets remaining and supply at zero, a later ERC-4626 deposit can round to zero shares and lose the deposit. The official script avoids zero supply by minting 100 shares to an unrecoverable address.

The permanent seed creates the opposite lifecycle defect: `receiveProfitDistribution` checks only whether `totalSupply` is zero. Dead shares make the check permanently false, so revenue sent before live TVL vests to unrecoverable shares. The seed also owns a permanent fraction of subsequent yield.

Evidence. One PoC demonstrated a zero-share follow-on deposit after the last live redemption with pending profit. A second demonstrated that dead seed shares make the `NoDepositors` guard accept and lock revenue.

Recommendation

Encode the lifecycle invariant onchain rather than relying on a script: distinguish live from seed supply, reject revenue until meaningful live TVL exists, prevent the final live redemption while pending profit exists, and minimize or eliminate economically participating dead shares.

LOW

STATUS: OPEN

L-02 - Revenue hooks are credited when a receiver pulls nothing

Affected. [Treasury.deliverRevenue](#).

The Treasury approves the selected receiver, calls its distribution hook, clears the approval, and credits the full nominal amount as paid without measuring the Treasury's actual USD8 balance change. A non-pulling receiver can return successfully while receiving zero, yet accounting and events report full distribution.

Impact and precondition. Funds remain in the Treasury rather than being stolen, but revenue accounting, monitoring, and downstream assumptions become false. Current Savings and pool hooks pull correctly; the issue requires a misconfigured or future receiver.

Evidence. A passing mock-receiver PoC returned normally without transferring USD8. The Treasury reported full distribution while the receiver balance remained zero.

Recommendation

Measure the Treasury's USD8 balance immediately before and after the hook. Require the exact expected delta or account only the observed amount, and revert on a partial or zero pull when exact delivery is required.

Trust model and assurance observations

The following observations are not included in the vulnerability counts because they are primarily centralization, configuration, documentation, or assurance concerns. They materially affect the production security model.

Observation	Security consequence	Recommended control
Fast admin can unpause	Registry.setPaused accepts both values, despite deployment comments describing a deny-only role.	Separate pause and unpause authorities; require timelock or multisig for unpause.
Beta root correction	adminCorrectSettlement can replace a settlement root within the correction flow, giving the beta admin custodial influence over payouts.	Publish the trust assumption, use a Safe, separate roles, monitor corrections, and sunset beta powers before material TVL.
USD8 pause does not gate mint/burn	A token-level pause is not a complete emergency stop; current safety depends on pausing Treasury paths.	Gate mint and burn or document and test the exact layered pause model.
Release config is incomplete	Offchain contract addresses are zero and feed configuration is empty, so the checked-in CLI is not a production release artifact.	Use chain-specific, reviewed manifests with code-hash and feed validation.
Signer and detection code absent	TEE event detection, key custody, enclave build, booster source, and oracle adapters could invalidate otherwise correct onchain assumptions.	Audit the complete production artifact, reproducible build, key ceremony, rotation, monitoring, and incident response.
Documentation mismatch	The offchain README uses a different eligibility cutoff and omits eligibleAmount from the documented Merkle leaf.	Generate protocol documentation from tested schemas and add independent verifier fixtures.
CI omits critical paths	Default CI runs Forge tests but not offchain build/tests, FFI integration, dependency audit, or legacy storage migrations.	Make all release-critical suites mandatory and archive compiler/storage metadata.
Generic asset assumptions	Decimal-offset ERC-4626 shares, rebasing tokens, upgradeable callback tokens, and non-log-complete assets can violate settlement recipes.	Require normalized adapters and explicit asset-admission invariants.

NO CLEAN BILL FOR THE COMPLETE SYSTEM

Because the production signer, detector, oracle adapters, and deployment records are absent, this repository review cannot establish end-to-end settlement authenticity or operational resilience.

Verification results

The existing suites were supplemented with adversarial harnesses. All executable correctness gates passed; the formatting check identified only existing drift in tracked files that were not modified by this audit.

Check	Result	Notes
Foundry baseline	PASS	403 passed, 0 failed, 3 skipped using offline mode
Settlement FFI integration	PASS	3 integration tests passed with RUN_INTEGRATION=1 and --ffi
Offchain Vitest	PASS	26 tests passed
TypeScript build	PASS	tsc completed without errors
Stateless fuzzing	PASS	4 targets at 10,000 runs each
Stateful invariants	PASS	2 invariants; 512 runs x 1,000 depth; 1,024,000 total calls
Forge coverage (aggregate)	MEASURED	86.36% lines; 85.31% statements; 73.83% branches; 86.98% functions
Production dependency audit	PASS	npm reported 0 production vulnerabilities
Runtime contract size	PASS	All implementations below the 24 KB limit; DefilInsurance 20,266 bytes
Forge lint	PASS WITH NOTES	Warnings and notes only
Forge formatting	DRIFT	Existing formatting differences in src/DefilInsurance.sol and test/AuditFindings.t.sol

PoC inventory

Finding	Reproduction or analysis
C-01	test_LegacyManagedToNamespacedBaseUpgradeBricksProxy
H-01	test_LastClaimRefundCallbackEscapesPoolHaircut
M-01	Lower-cap getLogs truncation followed by omitted balance outflow
M-02	Production settlement computation with high-score one-wei no-show claim
M-03	test_PriorSpendAndNextOpenCanOccurInSameBlock
M-04	Complexity and RPC-call amplification over adversarial registration sets
M-05	testDonationInflationLossPassesTolerance
M-06	Zero-redemption and new-minter dilution tests across Treasury rotation
L-01	Zero-supply pending-profit and dead-seed profit-lock tests
L-02	test_NonPullingReceiverIsReportedAsFullyDistributed

Repository state

The audit did not modify production repository files. At delivery, existing changes in README.md, src/SingleAssetCoverPool.sol, and test/SingleAssetCoverPool.t.sol remain preserved. The generated PDF is the only audit deliverable added under output/pdf.

Remediation roadmap

The ordering below is based on dependency and blast radius, not only severity. Storage history must be resolved before any upgrade; settlement integrity must be resolved before accepting coverage claims.

Phase	Required work	Exit criteria
P0 - Upgrade safety	Inventory live proxies and beacons; create legacy bridge migrations; define pool snapshot migration.	Fork tests prove legacy-to-bridge-to-current state, authorization, balances, liabilities, and future upgradeability.
P0 - Claim finalization	Keep incident freeze active through all external interactions; add cross-contract callback tests.	No callback can complete a pool exit before the final payout and score update finish.
P1 - Settlement integrity	Repair RPC completeness, capped allocation, score consumption, same-block snapshot, and anti-spam scaling.	Independent implementations produce identical roots under dense logs, caps, cancellation, sybils, and timing boundaries.
P1 - Capital safety	Replace base-unit tolerance; constrain vault admission; make Treasury migration atomic.	Donation, rounding, zero-reserve rotation, and fragmented-liability tests all revert or preserve value.
P2 - Lifecycle and reporting	Repair Savings live-supply lifecycle and receiver delivery accounting.	No zero-share deposit, unrecoverable pre-TVL yield, or false distribution accounting is possible.
P2 - Assurance	Complete production signer/config audit, narrow beta powers, align docs, and expand CI.	Reproducible release manifest binds source, bytecode, storage layout, addresses, feeds, signer measurement, and passing suites.

Recommended retest package

- Code diff and threat-model review for every finding remediation.
- Direct execution of the original PoCs against the patched implementations.
- Forked upgrade rehearsal against every discovered legacy deployment lineage.
- Independent offchain root computation across at least two complete data providers.
- End-to-end claim exercise using the actual production signer, detector, feeds, adapters, and governance roles.
- Release-candidate invariant and fuzz campaign with production configuration and realistic token mocks.

RELEASE GATE

Do not interpret passing baseline tests as closure. Each finding should be marked resolved only after its exploit PoC fails for the intended reason and the replacement invariant is explicitly tested.

End of report

USD8 Core Security Audit Report - final - 13 July 2026. Review target: eda7a22 plus current worktree.